

Инструменты локализации SS14 (SS14 RU Translation Tools)

Данная директория содержит набор Python-скриптов для автоматизации работы с переводами (Fluent `.ftl`), извлечения текста из прототипов (YAML) и интеграции с облачной платформой переводов Lokalise.

Быстрый старт (Основной инструмент)

Основным инструментом для ручной работы с переводами является **Менеджер локализации**. Он объединяет в себе извлечение текста, синхронизацию файлов, очистку от мусора и систему резервных копий.

Для работы просто запустите один из файлов (в зависимости от вашей ОС):

- `start_manager.bat` — для Windows.
- `start_manager.sh` — для Linux/macOS.

Скрипт автоматически проверит зависимости из `requirements.txt` и откроет консольное меню управления.

Описание файлов

Файлы в этой директории разделены на несколько логических модулей.

1. Управление и интерактивная работа

Файлы, предназначенные для непосредственного запуска пользователем.

- `locale_manager.py` Главный скрипт с интерактивным меню. Содержит логику синхронизации структуры (копирование новых ключей из английской локали в русскую), систему резервного копирования (`logs/` , `backups/`), инструмент пошагового ручного перевода и запуск полного конвейера обработки.
- `** start_manager.bat / start_manager.sh **` Скрипты-обертки для удобного запуска `locale_manager.py`. Автоматически устанавливают необходимые библиотеки перед запуском.
- `** translation.bat / translation.sh **` Устаревшие/альтернативные скрипты конвейера. Последовательно запускают извлечение YAML, очистку дубликатов и пустых файлов без создания резервных копий и вывода интерактивного меню.

2. Извлечение и очистка

Скрипты для автоматической работы с файлами проекта (могут вызываться отдельно или через менеджер).

- `yamlextractor.py` Проходит по всем `.yaml` файлам в папке `Prototypes` . Ищет поля `name` , `description` и `suffix` у сущностей, генерирует для них Fluent-ключи (например, `ent-ID-name`) и сохраняет их в английские `.ftl` файлы.
- `yamlmodels.py` Вспомогательный файл для `yamlextractor.py` . Содержит классы и структуру данных для правильного чтения YAML-файлов.
- `clean_duplicates.py` Проверяет русские `.ftl` файлы на наличие дублирующихся ключей перевода. Если находит повторы, оставляет один, а остальные удаляет. Логирует свои действия.
- `clean_empty.py` Сканирует директории с переводами и удаляет `.ftl` файлы нулевого размера, а также пустые папки, чтобы поддерживать чистоту структуры.

3. Интеграция с сервисом Lokalise

Скрипты для связи локальных файлов с платформой совместного перевода Lokalise.

- `lokalise_project.py` Отвечает за подключение к API Lokalise, загрузку ключей и переводов с сервера.
- `lokalisemodels.py` Описывает структуру данных (модели) для работы с ключами Lokalise и формирования путей к файлам.
- `lokalise_fluent_ast_comparer_manager.py` Связующее звено между AST (синтаксическим деревом Fluent) и Lokalise. Определяет, какие ключи нужно обновить в локальных файлах, какие добавить на сервер, а какие удалить.
- `translationsassembler.py` Сборщик переводов. Получает данные из Lokalise, сравнивает их с локальными `.ftl` файлами и применяет изменения, сериализуя их обратно в правильный Fluent-формат.

4. Ядро обработки Fluent (AST)

Модуль для глубокого синтаксического анализа `.ftl` файлов (Abstract Syntax Tree). Позволяет скриптам понимать Fluent-код как структуру, а не просто как текст.

- `fluentast.py` Базовые классы для работы с Fluent AST. Разбирает сообщения на ключи, значения, атрибуты и комментарии.
- `fluentastcomparer.py` Инструмент для сравнения двух синтаксических деревьев (например, английского и русского файла). Находит разницу: недостающие ключи, измененные атрибуты и т.д.
- `fluentastmanager.py` Управляет изменениями в дереве: обновляет узлы, вставляет новые переводы на нужные места.
- `fluentformatter.py` Форматтер кода. Проходит по всем файлам и выравнивает отступы, убирает лишние пробелы, приводя код к единому стилю (стайлгайду).

5. Системные и служебные файлы

- `file.py` Универсальная обертка над файловой системой. Обрабатывает чтение и запись файлов с правильной кодировкой (`utf-8`), обходит проблему со спецсимволом BOM и управляет путями.
- `project.py` Хранит базовую конфигурацию путей проекта. Скрипт сам “понимает”, где находится корневая папка игры (по маркеру `SpaceStation14.sln`) и выстраивает пути к `Locale` и `Prototypes` .
- `requirements.txt` Список Python-библиотек, необходимых для работы скриптов (`fluent` , `PyYAML` , `pydash` , `localise` и др.).
- `__init__.py` Системный файл Python. Указывает интерпретатору, что данная папка является модулем, что позволяет скриптам импортировать друг друга.

Требования

Для работы всех инструментов требуется:

- Установленный **Python 3.8** или выше.
- Зависимости из `requirements.txt` (устанавливаются автоматически при использовании `.bat` / `.sh` файлов).